

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: Task 5

Student Name: R Varshini

Reg. No.: A24126552115

Date: 21-09-2026

1. TITLE

Basic Express.js Student Server with Routing (/, /students and /about)

2. AIM

To create a basic Express.js server that provides three routes using the HTTP GET method: / (home), /**students** (a list of at least five students returned as JSON) and /**about** (information about the application), and to verify the responses in a browser.

3. OBJECTIVE

The objectives of this experiment are:

- To install Express.js in a Node.js project using npm.
- To create an Express application and start a server on a port.
- To define routes using `app.get()` with appropriate HTTP methods.
- To send a plain-text response using `res.send()`.
- To return structured data in JSON format using `res.json()`.
- To test the routes in a browser and verify the actual output against the expected output.

4. THEORY

Node.js is a JavaScript runtime that allows JavaScript to run outside the browser. Express.js is a minimal web framework built on Node.js that makes it easy to create servers, define routes and handle HTTP requests and responses.

- **Server:** a program that listens on a port and sends a response for every client request. Here the server listens on port 3000 of localhost.
- **require('express'):** loads the Express module that was installed with npm.
- **Routing:** mapping a URL path and an HTTP method to a handler function, written as `app.METHOD(path, handler)`.
- **HTTP GET:** the method used to retrieve data without changing it on the server. All three routes use `app.get()` because they only read data.
- **req and res:** `req` holds details of the incoming request and `res` is used to send the response back.
- **res.send():** sends a text (or HTML) response. **res.json():** converts a JavaScript object or array into JSON and sets the content type to `application/json`.
- **express.json():** middleware that parses JSON request bodies; it is included so that POST routes can be added later.
- **app.listen(PORT, callback):** starts the server; the callback runs once the server is ready.
- **404 Not Found:** if no route matches the request, Express replies with "Cannot GET /path" and status code 404.

Routes implemented:

Route	Method	Purpose	Response
/	GET	Home page	Text: Welcome to the Student Server!
/students	GET	Student list	JSON array of 5 students (id, name)
/about	GET	Application information	Text describing the application

Flow: Browser request (GET URL) → Express matches the route → handler function runs → `res.send()` / `res.json()` → response displayed in the browser

5. ALGORITHM / PROCEDURE

1. Create a project folder and open a terminal in it.
2. Install Express using the command `npm install express`.
3. Create the file `pbm2.js` and import Express with `require('express')`.
4. Create the application object `app = express()` and set `PORT = 3000`.
5. Add the `express.json()` middleware.
6. Define the `GET /` route to send a welcome message.
7. Define the `GET /students` route with an array of five student objects (id and name) and return it using `res.json()`.
8. Define the `GET /about` route to send information about the application.
9. Start the server using `app.listen(PORT, ...)` and print a message in the console.
10. Run the server with `node pbm2.js`.
11. Open the browser and visit `/`, `/students` and `/about`; also visit an undefined route to check the 404 response.
12. Verify the actual output against the expected output for each test case.

6. PROGRAM

Commands used

```
npm install express
node pbm2.js
```

pbm2.js

```
// Import Express
const express = require('express');
const app = express();
const PORT = 3000;

// Middleware to parse JSON (optional if you want to handle POST requests later)
app.use(express.json());

// Route: Home
app.get('/', (req, res) => {
  res.send('Welcome to the Student Server!');
});

// Route: Students
app.get('/students', (req, res) => {
  const students = [
    { id: 1, name: 'Alice' },
    { id: 2, name: 'Bob' },
    { id: 3, name: 'Charlie' },
    { id: 4, name: 'David' },
    { id: 5, name: 'Eva' }
  ];
  res.json(students);
});

// Route: About
app.get('/about', (req, res) => {
  res.send('This is a basic Express.js application that manages student data and demonstrates routing.');
```

```
});

// Start the server
app.listen(PORT, () => {
  console.log(`🚀 Server is running at http://localhost:${PORT}`);
});
```

7. CODE EXPLANATION

Code	Explanation
<code>require('express')</code>	Imports the Express framework into the program.
<code>const app = express();</code>	Creates the Express application object used to define routes.
<code>const PORT = 3000;</code>	Port number on which the server listens for requests.
<code>app.use(express.json());</code>	Middleware that parses JSON request bodies (useful for future POST requests).
<code>app.get('/', ...)</code>	Handles a GET request to the home route.
<code>res.send('Welcome ...')</code>	Sends a plain-text welcome message as the response.
<code>app.get('/students', ...)</code>	Handles a GET request for the student list.
<code>const students = [...]</code>	Array of five objects, each having an id and a name.
<code>res.json(students);</code>	Converts the array to JSON and sends it with content type application/json.
<code>app.get('/about', ...)</code>	Handles a GET request for information about the application.
<code>app.listen(PORT, ...)</code>	Starts the server on port 3000; the callback runs when the server is ready.
<code>console.log(`... \${PORT}`)</code>	Prints the server-running message with the port number in the terminal.

8. TEST CASES AND EXECUTION DATA

Each test case was executed by starting the server and sending the request below from the browser. The results are the responses displayed.

Test Case	Request	Expected Result	Actual Result	Status
TC-01	node pbm2.js	Server starts and prints the running message with the URL	Server is running at http://localhost:3000	Pass
TC-02	GET /	Welcome to the Student Server!	Welcome to the Student Server!	Pass
TC-03	GET /students	JSON array of 5 students: ids 1-5 (Alice, Bob, Charlie, David, Eva)	JSON array with 5 objects (id, name) displayed	Pass
TC-04	GET /about	Text describing the application	This is a basic Express.js application that manages student data and demonstrates routing.	Pass
TC-05	GET /contact (undefined route)	Cannot GET /contact (HTTP 404)	Cannot GET /contact	Pass

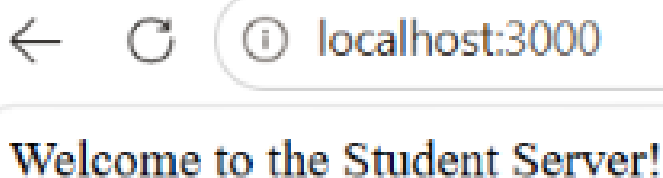
9. OUTPUT

Output 1: TC-01 - Installing Express and starting the server

```
PS C:\Users\varsh\OneDrive\Desktop\html> npm install express
added 68 packages in 4s
28 packages are looking for funding
  run `npm fund` for details
PS C:\Users\varsh\OneDrive\Desktop\html> node pbm2.js
🚀 Server is running at http://localhost:3000
```

npm installed Express (68 packages) and node pbm2.js printed the message that the server is running at http://localhost:3000.

Output 2: TC-02 - GET / (home route)



← ↻ ⓘ localhost:3000

Welcome to the Student Server!

The home route displayed **Welcome to the Student Server!**

Output 3: TC-03 - GET /students (list of five students)



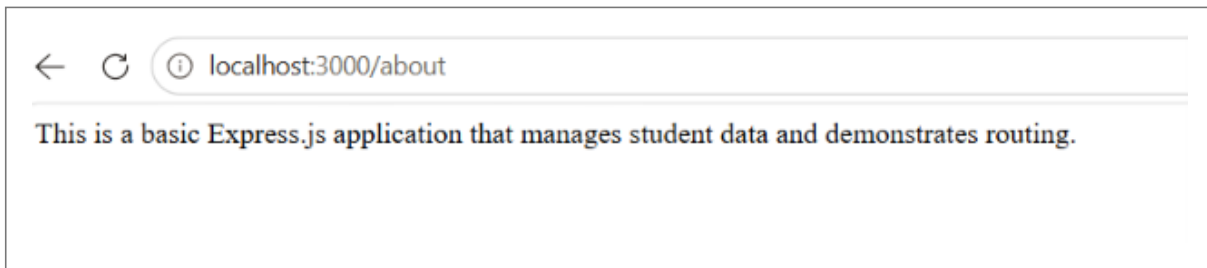
← ↻ ⓘ localhost:3000/students

Pretty-print ☒

```
[
  {
    "id": 1,
    "name": "Alice"
  },
  {
    "id": 2,
    "name": "Bob"
  },
  {
    "id": 3,
    "name": "Charlie"
  },
  {
    "id": 4,
    "name": "David"
  },
  {
    "id": 5,
    "name": "Eva"
  }
]
```

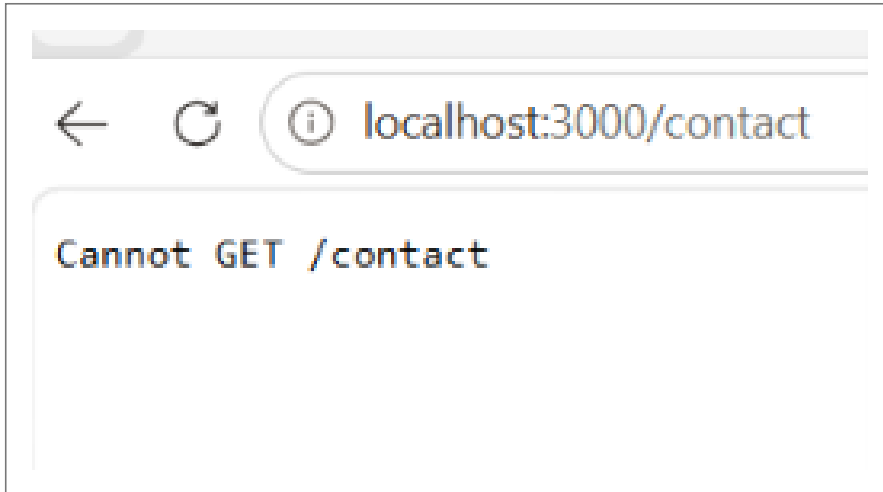
The /students route returned a JSON array of five students (ids 1 to 5).

Output 4: TC-04 - GET /about



The /about route displayed the description of the application.

Output 5: TC-05 - GET /contact (undefined route)



An undefined route returns **Cannot GET /contact** with status 404.

10. RESULT

Thus, a basic Express.js server was successfully created with the routes /, /students and /about using the HTTP GET method. The home route returned a welcome message, the /students route returned a JSON list of five students, and the /about route returned information about the application. The application was verified using multiple test cases.